



北京大學
PEKING UNIVERSITY

Visual Recognition – Part 2: Deep Learning Era

Mu Yadong

Wangxuan Institute of Computer Technology
Peking University

**Several slides are adapted from related courses or tutorials.
Internal use only. Please do not distribute the slides.**

Outline

- **Why DL suddenly works? (AlexNet, 2012)**
- **Can it go deeper? The development from AlexNet (2012) to Resnet (2015)**
- **Further extensions (DenseNet DualPathNet etc.)**
- **Network Understanding**

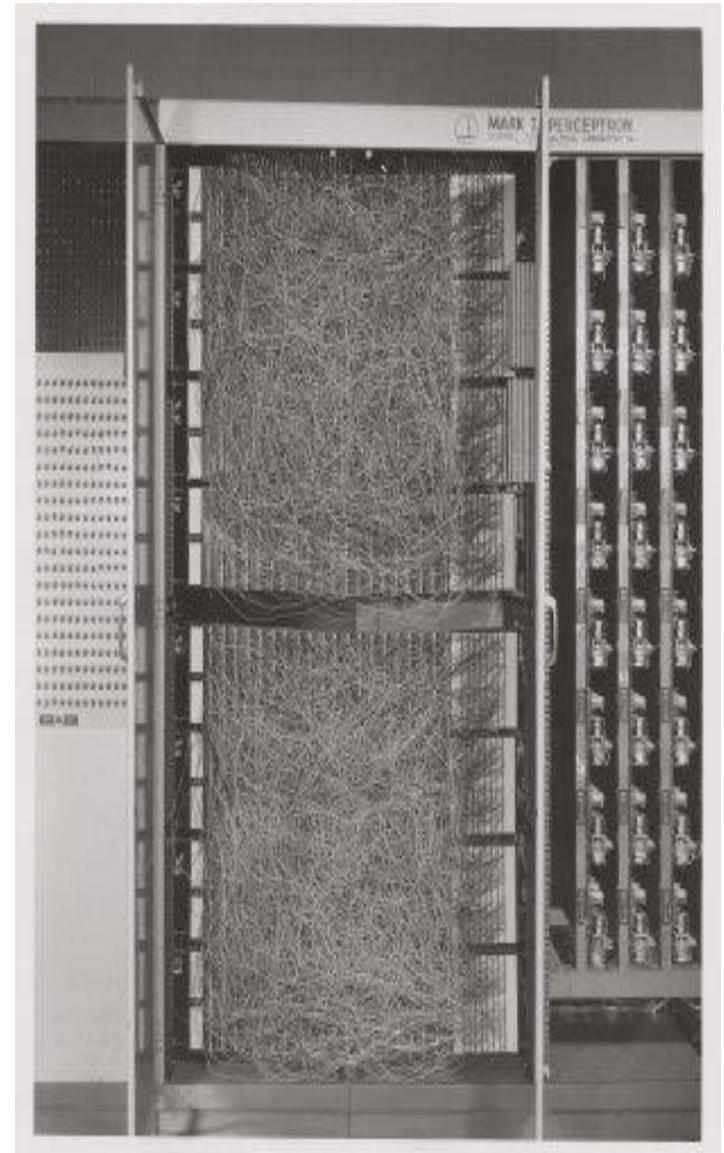
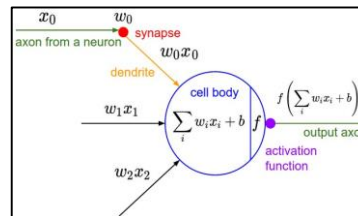
Perceptron

- The Mark I Perceptron machine was the first implementation of the perceptron algorithm.
- The machine was connected to a camera that used 20×20 cadmium sulfide photocells to produce a 400-pixel image.
- recognized letters of the alphabet

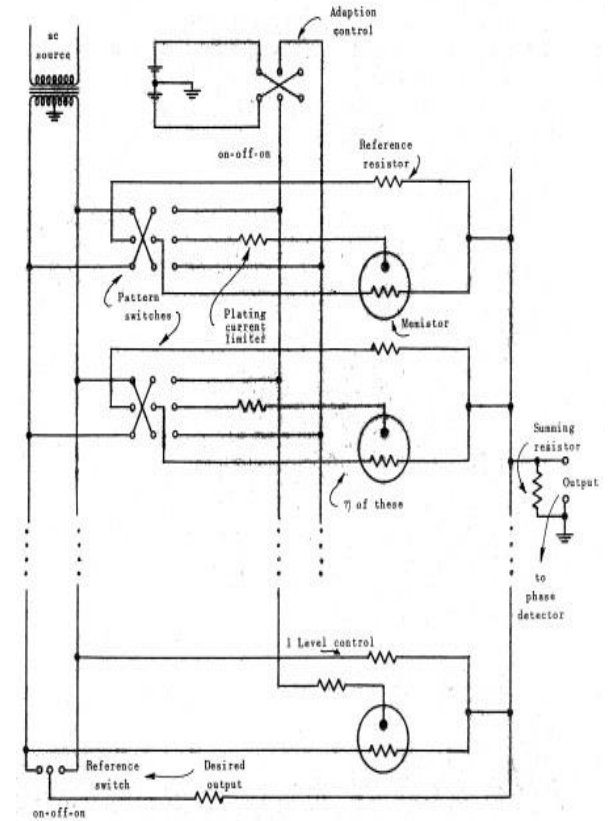
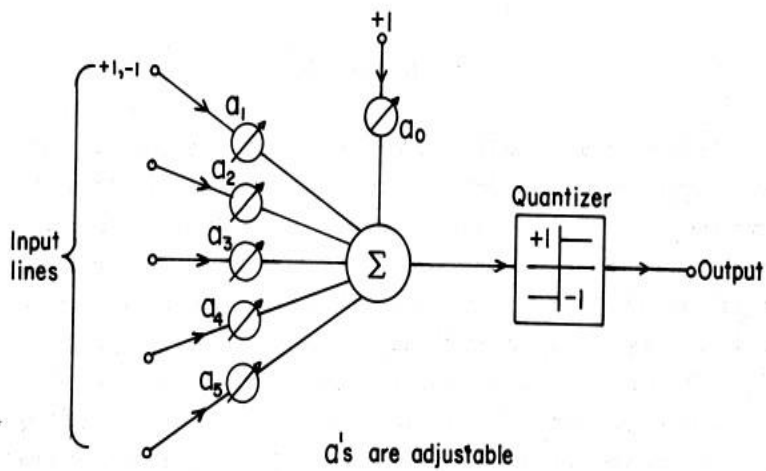
$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

update rule:

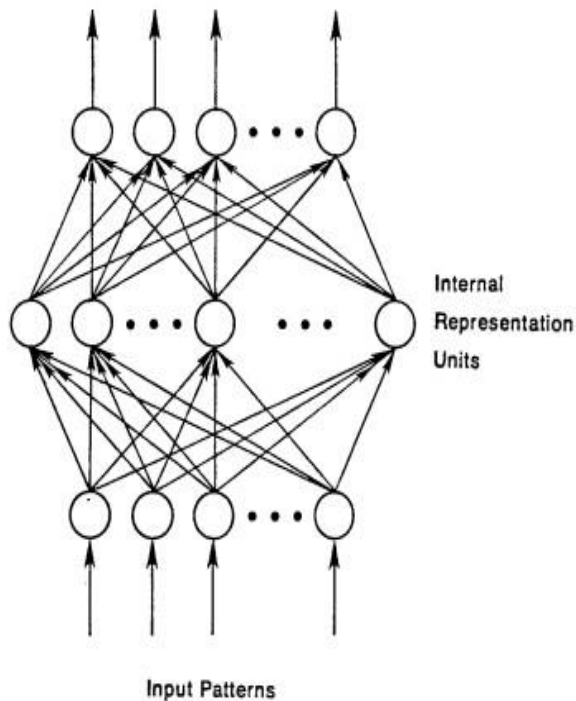
$$w_i(t+1) = w_i(t) + \alpha(d_j - y_j(t))x_{j,i}$$



Frank Rosenblatt, ~1957: Perceptron



Widrow and Hoff, ~1960: Adaline/Madaline



To be more specific, then, let

$$E_p = \frac{1}{2} \sum_j (t_{pj} - o_{pj})^2 \quad (2)$$

be our measure of the error on input/output pattern p and let $E = \sum E_p$ be our overall measure of the error. We wish to show that the delta rule implements a gradient descent in E when the units are linear. We will proceed by simply showing that

$$-\frac{\partial E_p}{\partial w_{ji}} = \delta_{pj} i_{ji},$$

which is proportional to $\Delta_p w_{ji}$ as prescribed by the delta rule. When there are no hidden units it is straightforward to compute the relevant derivative. For this purpose we use the chain rule to write the derivative as the product of two parts: the derivative of the error with respect to the output of the unit times the derivative of the output with respect to the weight.

$$\frac{\partial E_p}{\partial w_{ji}} = \frac{\partial E_p}{\partial o_{pj}} \frac{\partial o_{pj}}{\partial w_{ji}}. \quad (3)$$

The first part tells how the error changes with the output of the j th unit and the second part tells how much changing w_{ji} changes that output. Now, the derivatives are easy to compute. First, from Equation 2

$$\frac{\partial E_p}{\partial o_{pj}} = -(t_{pj} - o_{pj}) = -\delta_{pj}. \quad (4)$$

Not surprisingly, the contribution of unit u_j to the error is simply proportional to δ_{pj} . Moreover, since we have linear units,

$$o_{pj} = \sum_i w_{ji} i_{pi}, \quad (5)$$

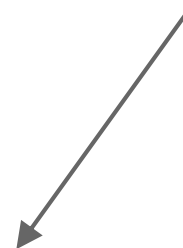
from which we conclude that

$$\frac{\partial o_{pj}}{\partial w_{ji}} = i_{pi}.$$

Thus, substituting back into Equation 3, we see that

$$-\frac{\partial E_p}{\partial w_{ji}} = \delta_{pj} i_{pi} \quad (6)$$

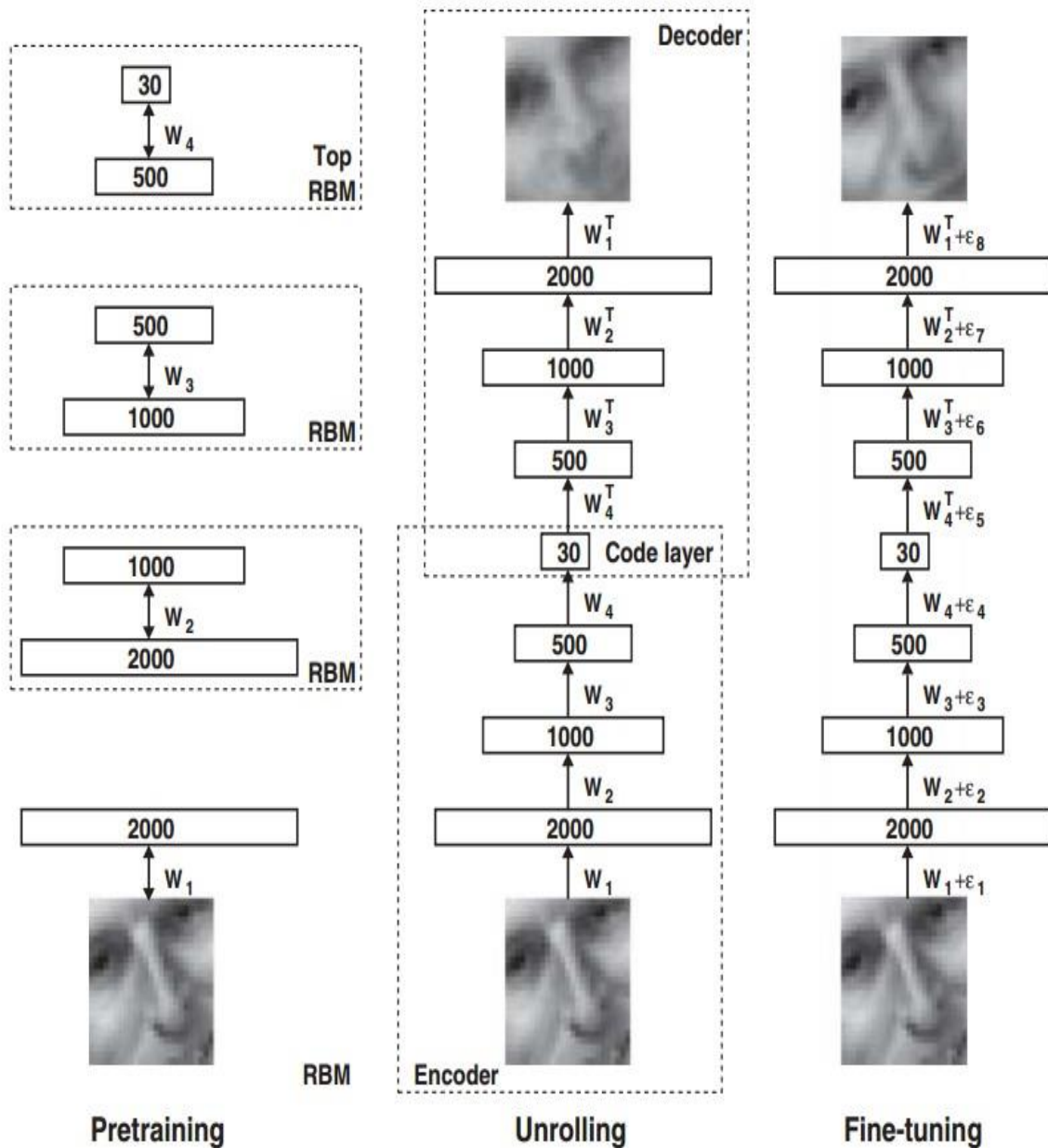
recognizable math



Rumelhart et al. 1986: First time back-propagation became popular

[Hinton and Salakhutdinov 2006]

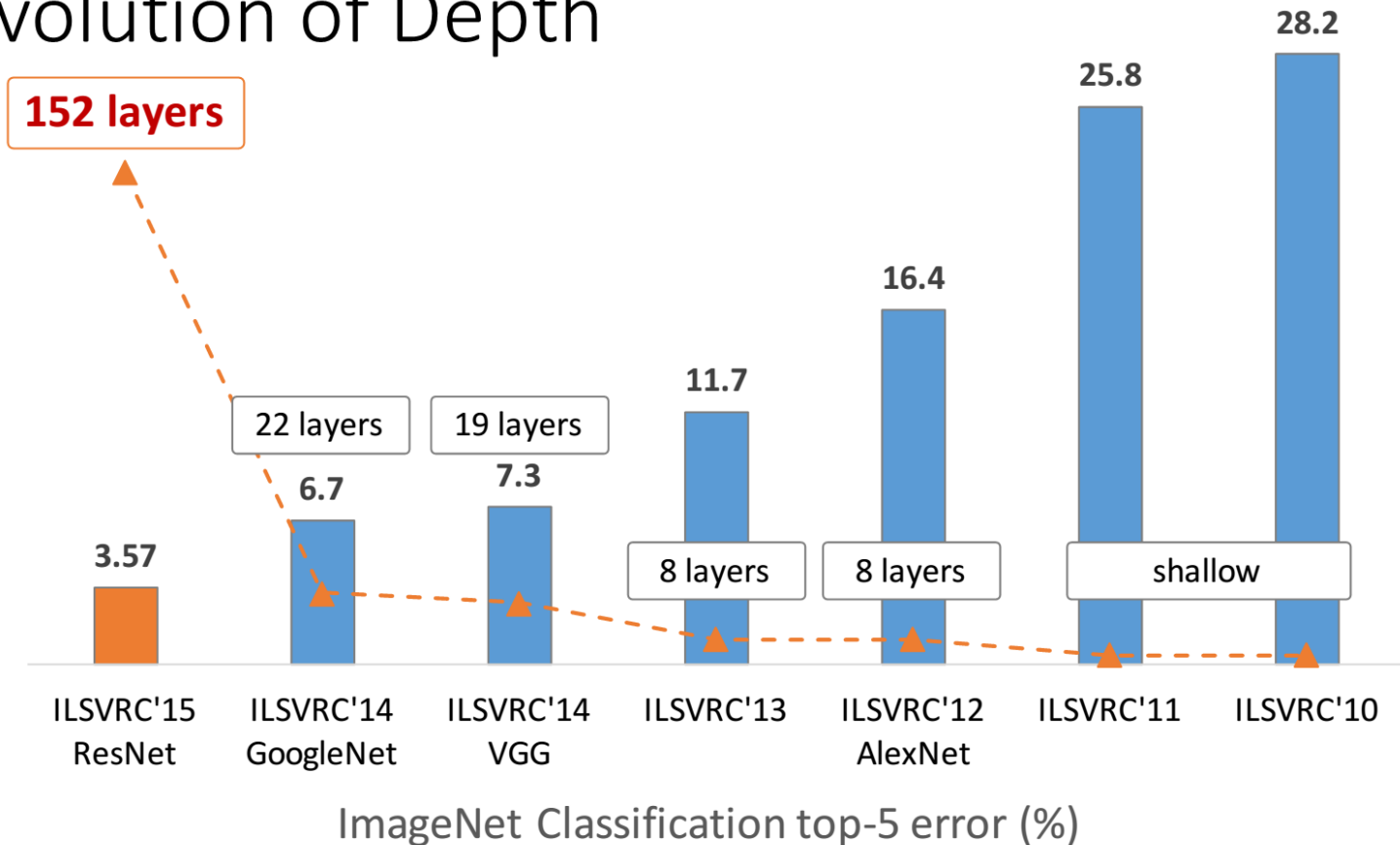
Reinvigorated research in Deep Learning



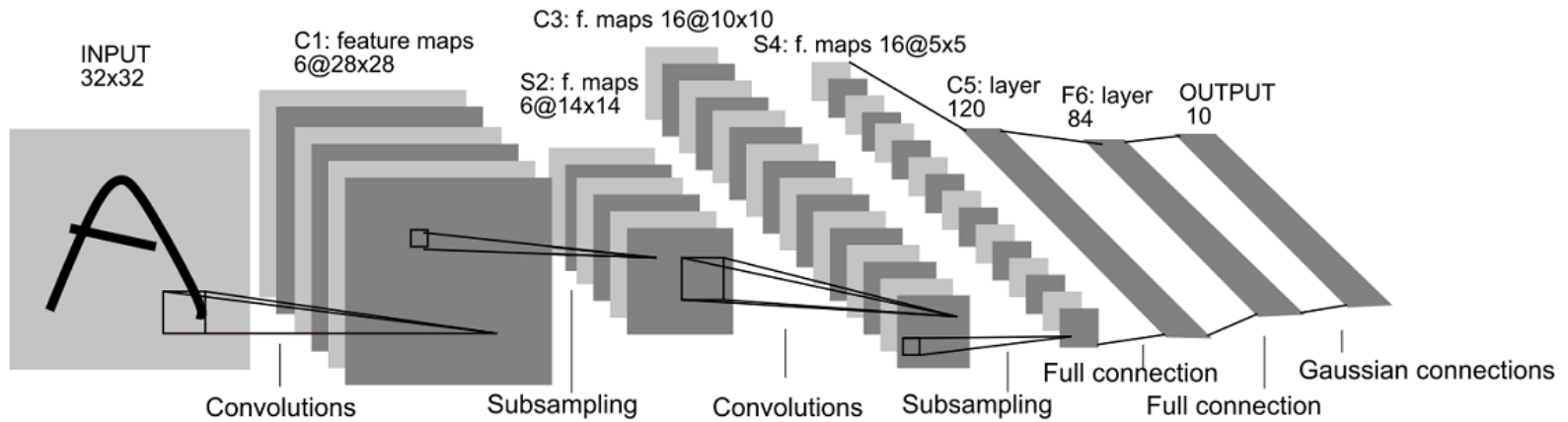
AlexNet

- Named after Alex Krizhevsky, proposed in 2012

Revolution of Depth



LeNet-5

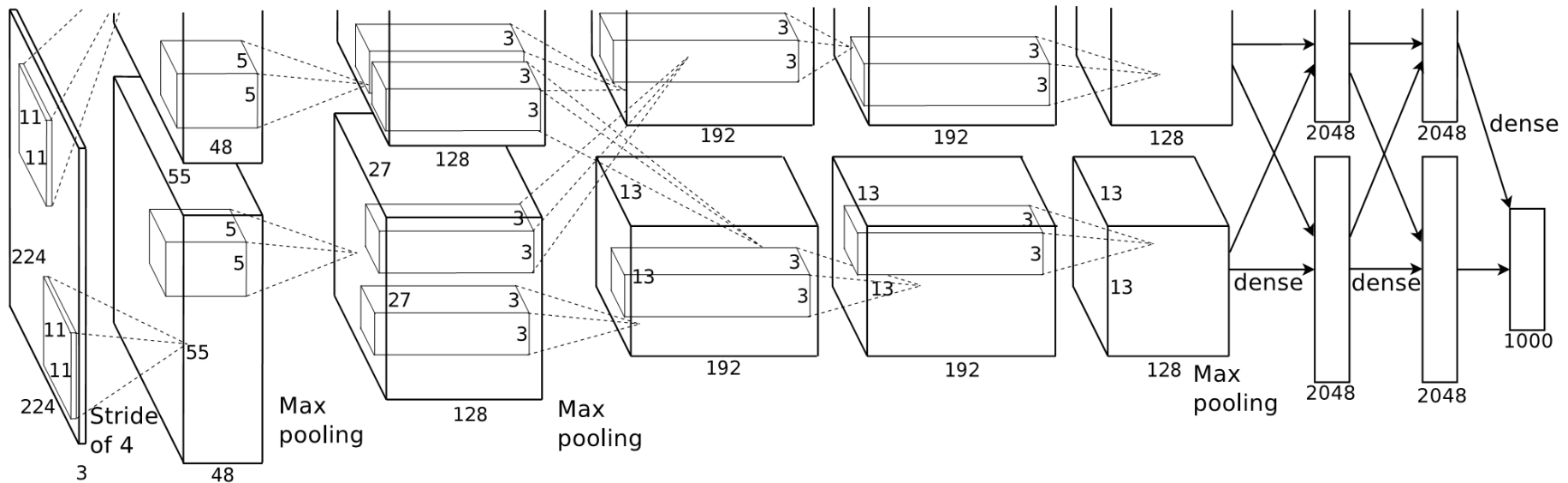


- Input: 32x32 pixel image. Largest character is 20x20
(All important info should be in the center of the receptive field of the highest level feature detectors)
- Cx: Convolutional layer
- Sx: Subsample layer
- Fx: Fully connected layer
- Black and White pixel values are normalized:
E.g. White = -0.1, Black = 1.175 (Mean of pixels = 0, Std of pixels = 1)
14

Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. Proceedings of the IEEE, november 1998.

AlexNet

- Much larger than LeNet-5
- Trained on two GTX 580 GPU
- Largest networks at its time
- Utilize multiple engineering tricks (dropout, ReLU)



Alex Krizhevsky et al., ImageNet Classification with Deep Convolutional Neural Networks, NIPS 2012

Why DL Suddenly works?

...It may be that the primary barriers to the success of neural networks were psychological (practitioners did not expect neural networks to work, so they did not make a serious effort to use neural networks)...

-- Goodfellow et al. "deep Learning"

Why DL Suddenly works? – my biased view

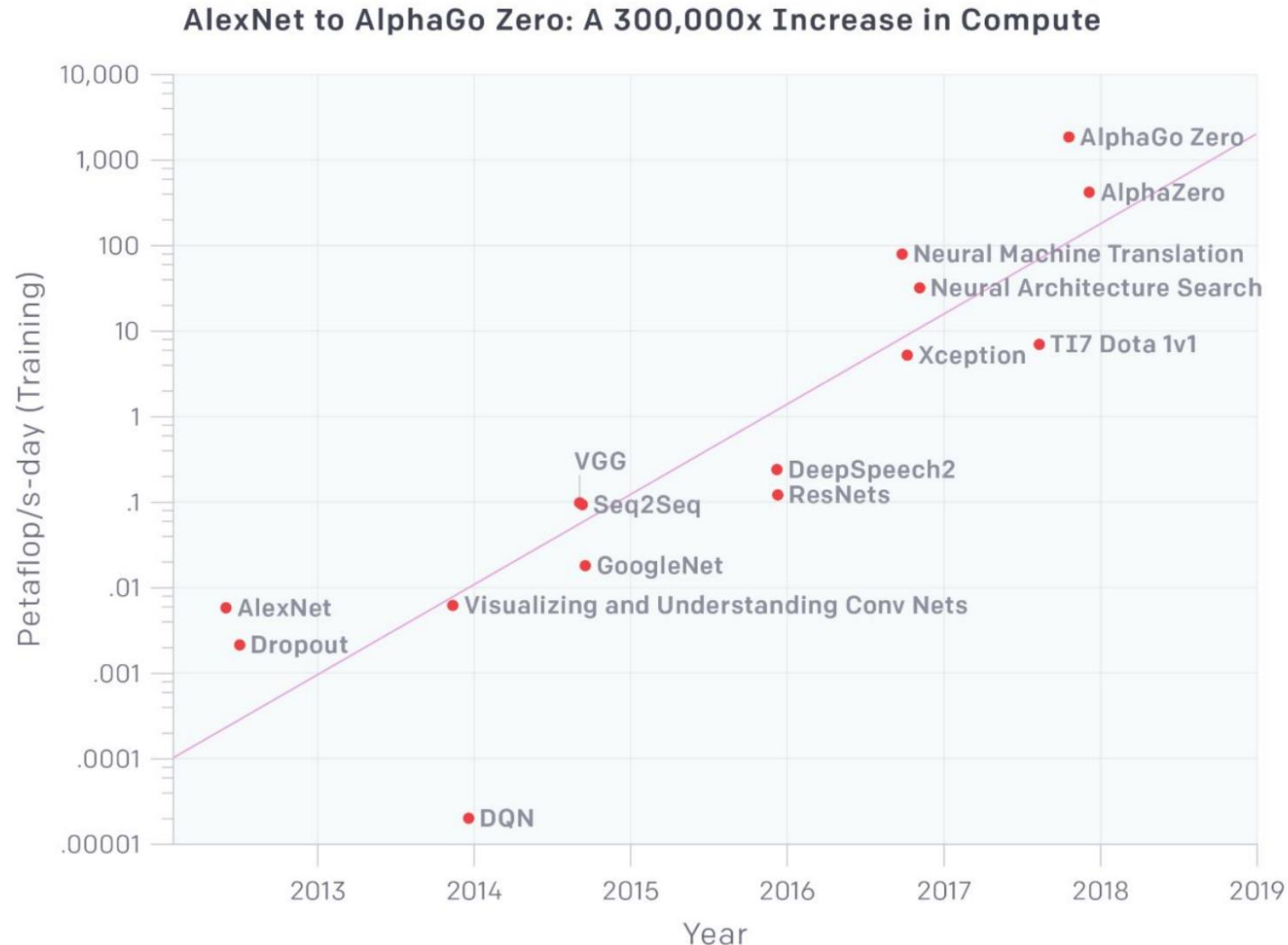
- Emerging of big visual data: camera, social media, cloud, 3/4G
- GPU -> large network
- New engineering tricks (dropout, ReLU etc.)
- Industrial applications

所有的产品，特别是基于人工智能的产品，都有这样一个特点。技术所能达到的性能的上界（简称技术上界），如果不能超过用户使用时对性能要求的下界（简称使用下界），那么这个产品就很难被用户接受，其实用化也基本不可能成功。也就是说，技术所能达到的性能的绝对高低并不重要，重要的是它必须高于用户使用时所能容忍的底线。



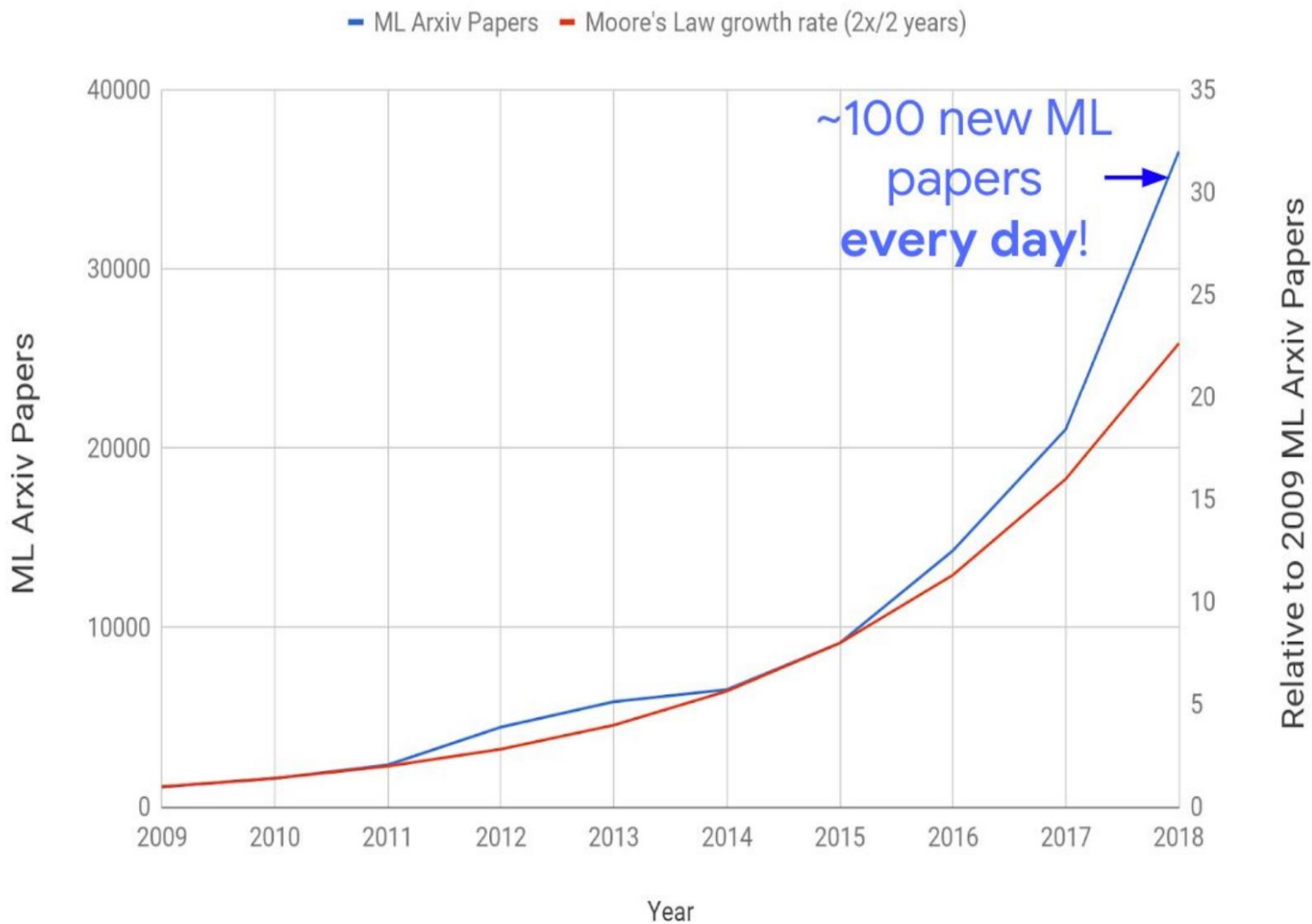
Li Hang (ByteDance)

AI Advances and their Computational Requirements



(Source: openai.com/blog/ai-and-compute/)

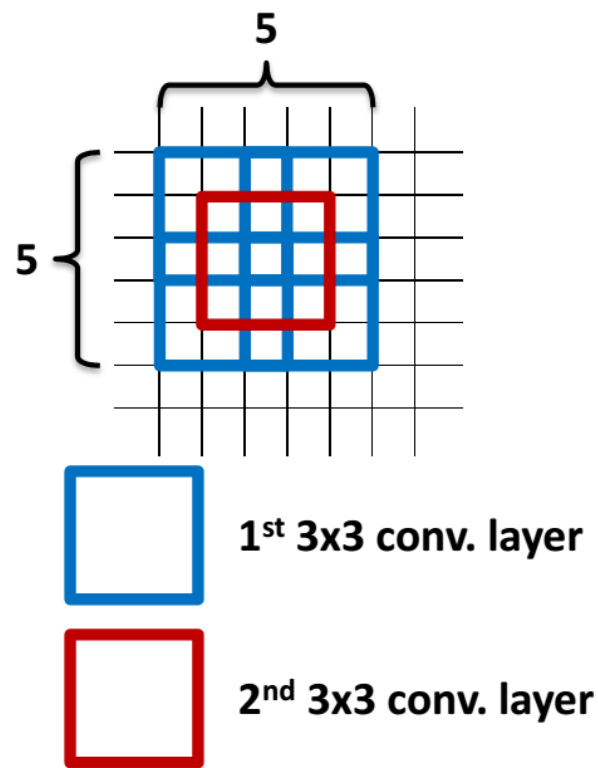
Expanding Academic/Industrial Communities



VGG Net

Why 3x3 layers?

- Stacked conv. layers have a large receptive field
 - two 3x3 layers – 5x5 receptive field
 - three 3x3 layers – 7x7 receptive field
- More non-linearity
- Less parameters to learn
 - ~140M per net



Network Design

Key design choices:

- 3x3 conv. kernels – very small
- conv. stride 1 – no loss of information

Other details:

- Rectification (ReLU) non-linearity
- 5 max-pool layers (x2 reduction)
- no normalisation
- 3 fully-connected (FC) layers

image

conv-64

conv-64

maxpool

conv-128

conv-128

maxpool

conv-256

conv-256

maxpool

conv-512

conv-512

maxpool

conv-512

conv-512

maxpool

FC-4096

FC-4096

FC-1000

softmax

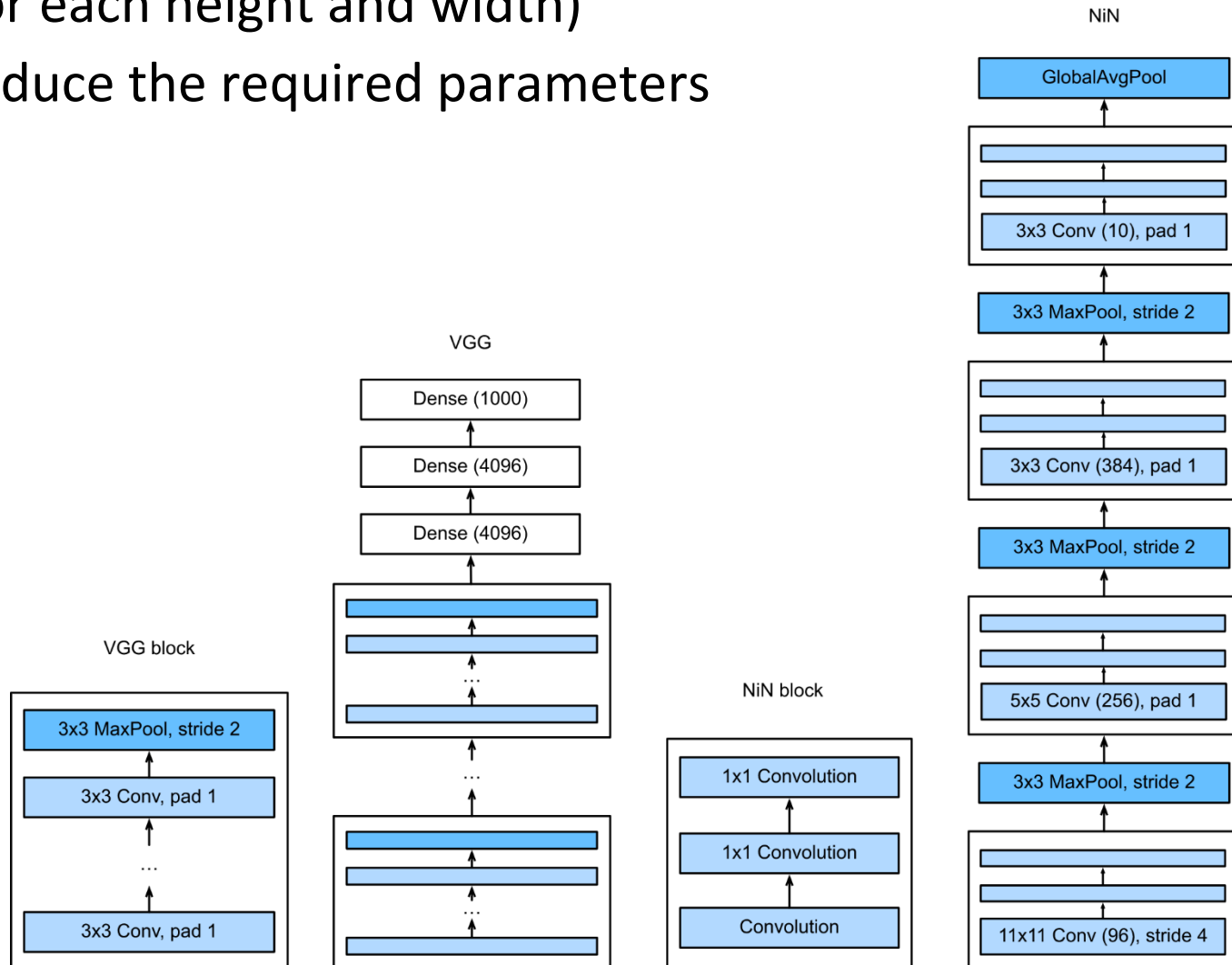
AlexNet v.s. VGG Net

- Sequentially connect *building blocks*

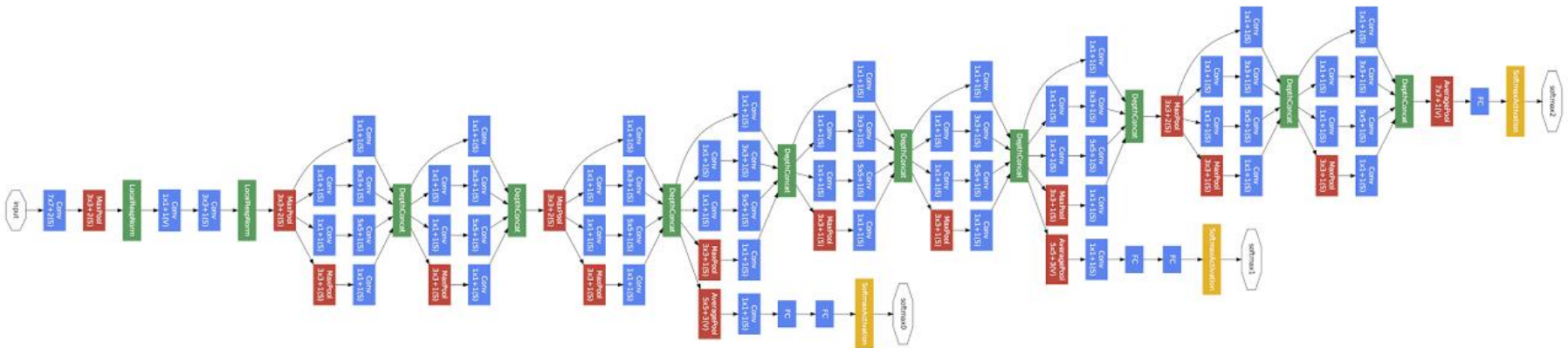


Network-in-Network (NiN) [lin et al. 2013]

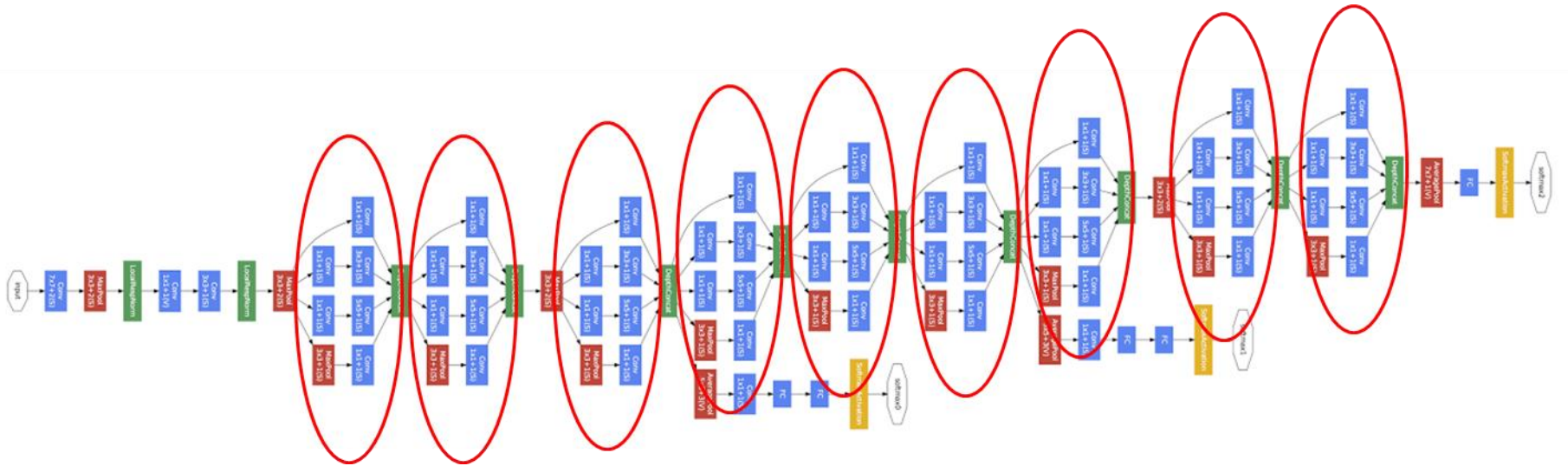
- Key idea: apply a fully-connected layer at each pixel location (for each height and width)
 - Reduce the required parameters
- 
- ```
graph BT; Input[] --> GAP[GlobalAvgPool]; GAP --> NiN[NiN]
```



# GoogLeNet



Szegedy, C. et al., Going deeper with convolutions. CVPR 2015.

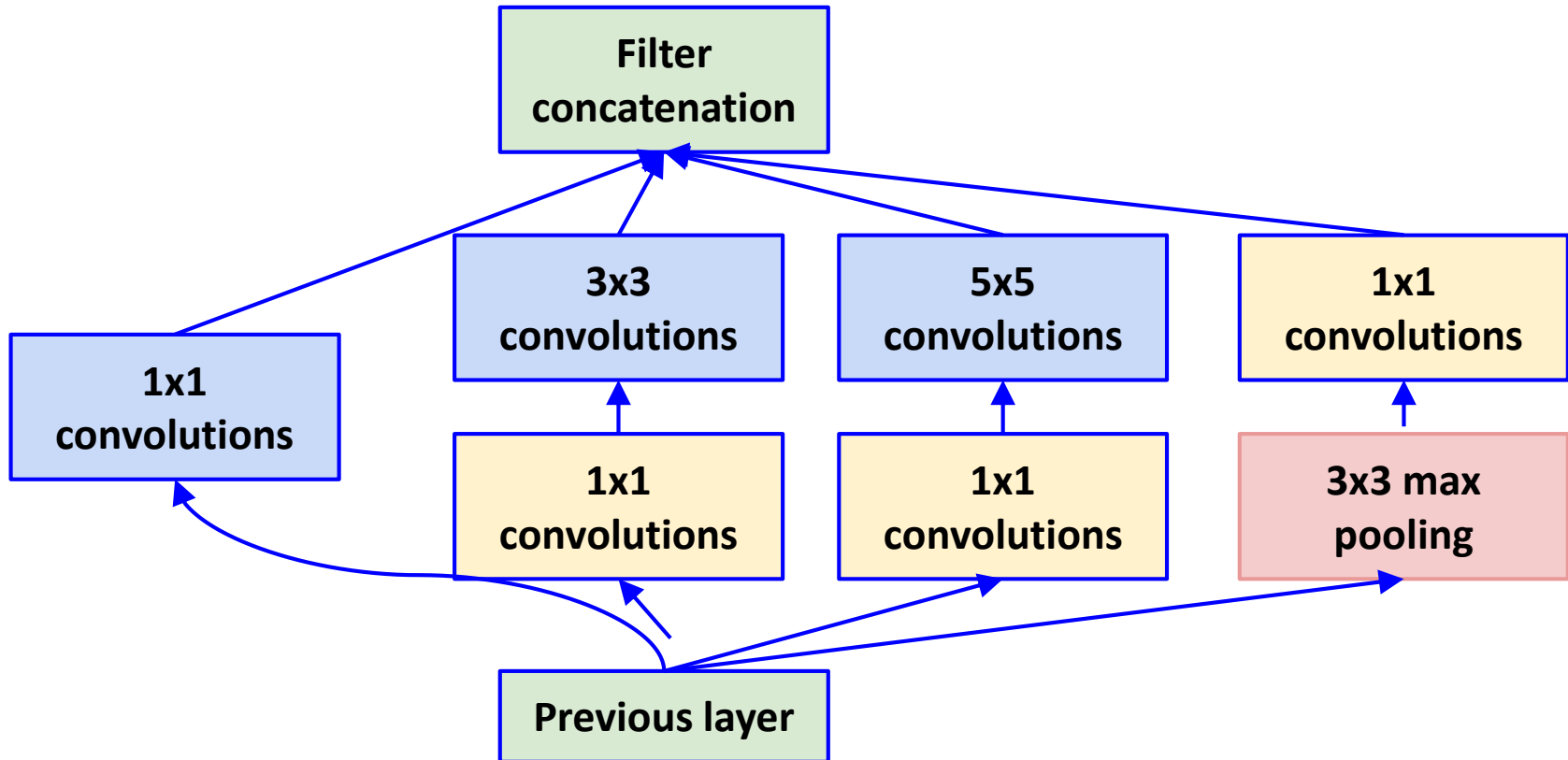


# Inception

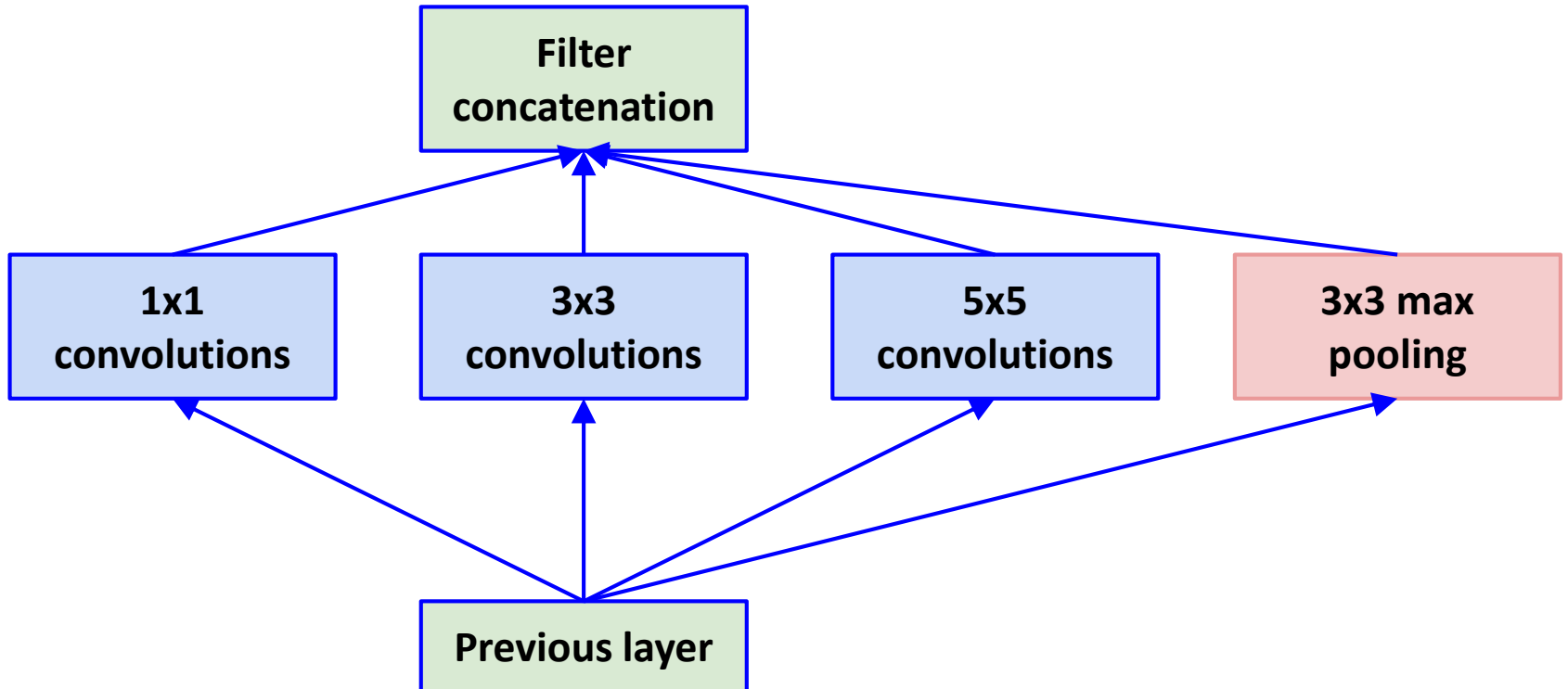
Network in a network in a network...

Convolution  
Pooling  
Softmax  
Other

# Inception module



# Naive idea (does not work!)



# ResNet

- See He Kaiming's ICML tutorial

# ResNet Variants

| layer name | output size | 18-layer                                                                  | 34-layer                                                                  | 50-layer                                                                                     | 101-layer                                                                                     | 152-layer                                                                                     |
|------------|-------------|---------------------------------------------------------------------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| conv1      | 112×112     | 7×7, 64, stride 2                                                         |                                                                           |                                                                                              |                                                                                               |                                                                                               |
| conv2_x    | 56×56       | 3×3 max pool, stride 2                                                    |                                                                           |                                                                                              |                                                                                               |                                                                                               |
|            |             | $\begin{bmatrix} 3\times 3, 64 \\ 3\times 3, 64 \end{bmatrix} \times 2$   | $\begin{bmatrix} 3\times 3, 64 \\ 3\times 3, 64 \end{bmatrix} \times 3$   | $\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$    | $\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$     | $\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$     |
| conv3_x    | 28×28       | $\begin{bmatrix} 3\times 3, 128 \\ 3\times 3, 128 \end{bmatrix} \times 2$ | $\begin{bmatrix} 3\times 3, 128 \\ 3\times 3, 128 \end{bmatrix} \times 4$ | $\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$  | $\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$   | $\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 8$   |
| conv4_x    | 14×14       | $\begin{bmatrix} 3\times 3, 256 \\ 3\times 3, 256 \end{bmatrix} \times 2$ | $\begin{bmatrix} 3\times 3, 256 \\ 3\times 3, 256 \end{bmatrix} \times 6$ | $\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 6$ | $\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 23$ | $\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 36$ |
| conv5_x    | 7×7         | $\begin{bmatrix} 3\times 3, 512 \\ 3\times 3, 512 \end{bmatrix} \times 2$ | $\begin{bmatrix} 3\times 3, 512 \\ 3\times 3, 512 \end{bmatrix} \times 3$ | $\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$ | $\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$  | $\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$  |
|            | 1×1         | average pool, 1000-d fc, softmax                                          |                                                                           |                                                                                              |                                                                                               |                                                                                               |
| FLOPs      |             | $1.8 \times 10^9$                                                         | $3.6 \times 10^9$                                                         | $3.8 \times 10^9$                                                                            | $7.6 \times 10^9$                                                                             | $11.3 \times 10^9$                                                                            |

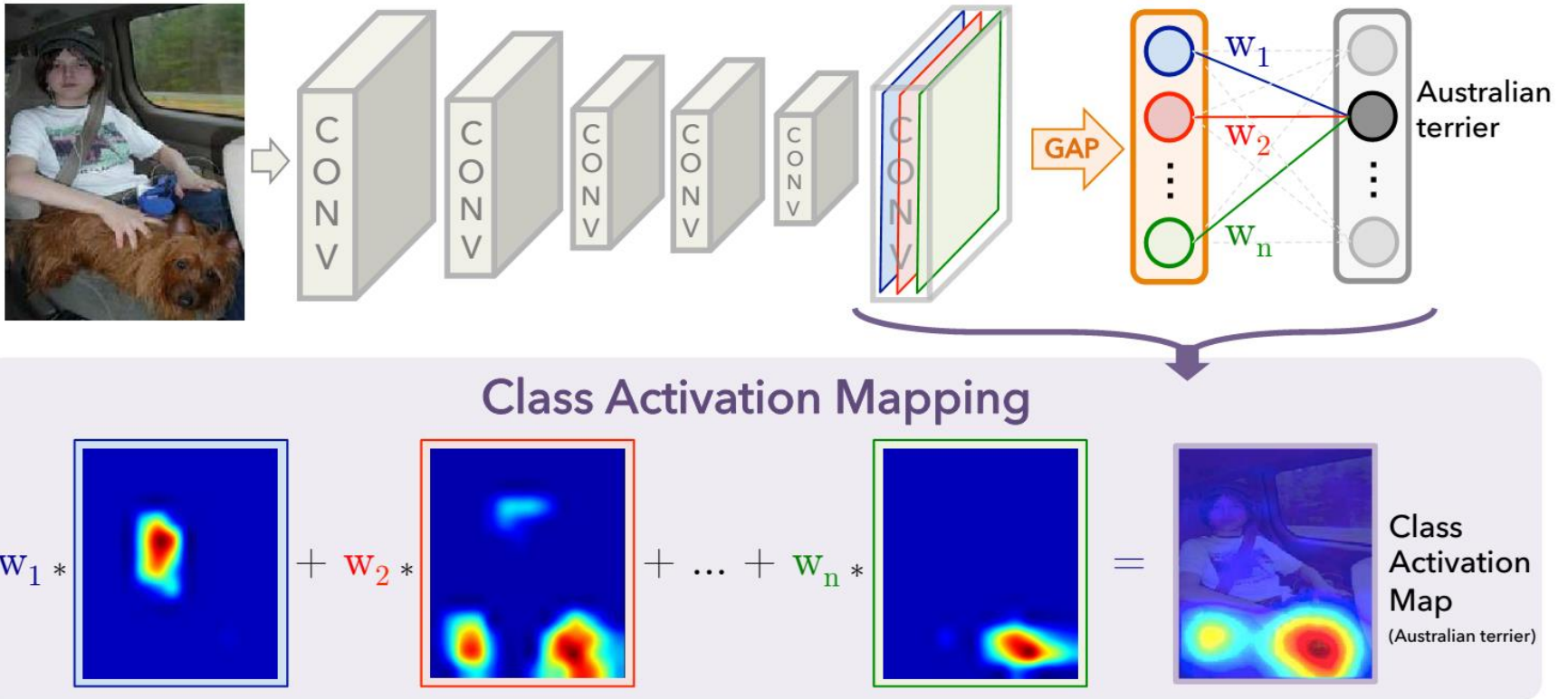
# DenseNet

- See DenseNet's CVPR slides



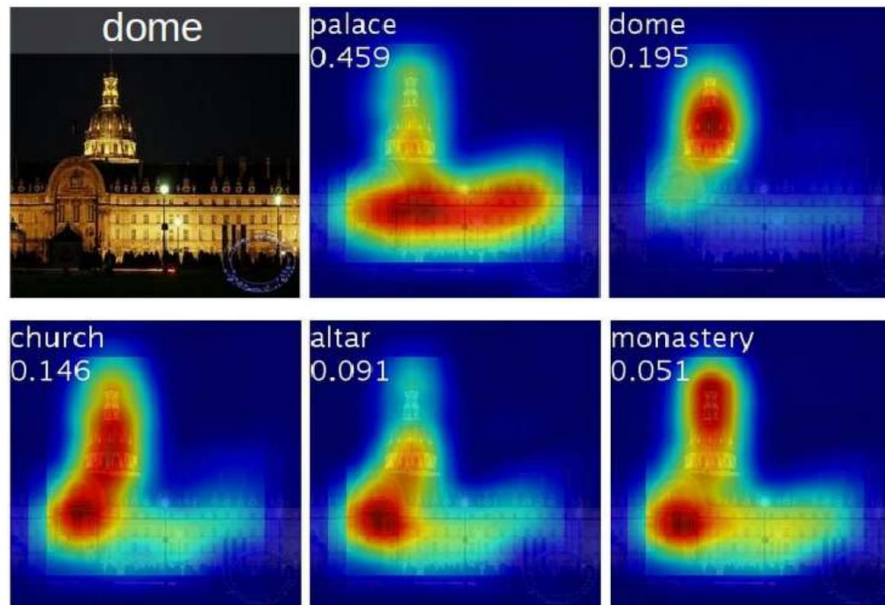
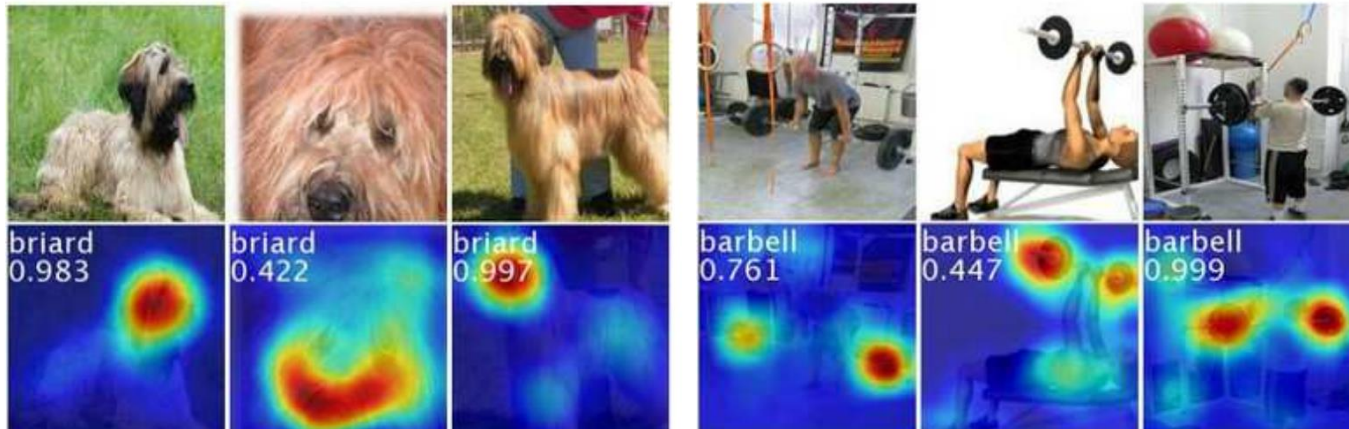
# Class Activation Map (CAM)

- Global Average Pooling



# Class Activation Map (CAM)

- Examples



**Questions?**